

Tic Tac Toe

A machine-coding / LLD walkthrough — plus the “add a computer player” extension

The second classic LLD problem, built the same disciplined way: **clarify** → **identify entities** → **build bottom-up** → **let patterns emerge**. The twist here is instructive — the base problem needs **no design patterns at all**. Forcing one in would be over-engineering. Then the interviewer says “*add a computer player*”, real behavioural variation appears, and **Strategy** finally earns its place.

What's inside

1. The game & the clarifying questions
2. Requirements (locked scope)
3. Entities & the ownership tree
4. Building the bricks: Symbol • Player • Board • Game
5. The win check — rows, columns, both diagonals (N×N)
6. Full runnable code
7. Extension — “add a computer player” with Strategy
8. The lesson: when NOT to use a pattern

1. The Game & Clarifying Questions

Tic Tac Toe: a 3×3 grid, two players (X and O) alternate placing marks on empty cells. First to get three in a row — horizontal, vertical or diagonal — wins. All nine cells full with no winner = draw.

```
X | O | X
-----
O | X | O
-----
|  | X   <- X wins (main diagonal)
```

The rules are famous, so what's left to clarify? Good candidates find the **hidden flexibility**:

Question	Interviewer's answer
Board size — always 3×3?	make it N×N, configurable. Default 3.
Always 2 players, X and O?	2 players; symbols configurable
Win condition?	N in a row — full row, column, or diagonal
Human vs computer?	human vs human for now; keep it extensible
Invalid moves?	reject; the SAME player retries

Why “N×N” matters: it's a trap for lazy candidates. Hardcode 3 everywhere and “now make it 5×5” means a rewrite. Design for the **variable**, not the specific number — the same lesson as “N floors” in Parking Lot.

2. Requirements (Locked Scope)

- N×N board (default 3), configurable
- 2 players, each with a configurable symbol (X, O)
- Players alternate turns, placing their symbol on an empty cell
- Win = N marks in a row / column / diagonal
- Draw = board full, no winner
- Invalid move (taken or out-of-bounds) → reject, same player retries
- Human vs human (extensible to a bot later)

3. Entities & Ownership Tree

Pull the nouns out of the scope. It's a small problem — expect about four core entities.

Entity	What it is
Game	the manager; runs turns, checks win/draw
Board	the N×N grid; holds marks, checks lines
Player	a player with a name + symbol
Symbol (enum)	X / O / EMPTY
Cell (optional)	one square — usually not needed; a 2D array suffices

The easy one to miss: the **Game** manager. After listing the physical things (board, players, symbols), always ask — “*who's in charge? who runs the loop?*” That question surfaces the controller class every time. (Same role ParkingLot played last problem.)

Note: “**turn**” feels like an entity but isn't — it's a *variable* inside Game tracking the current player index, not a class.

The ownership tree

```
Game
|-- has a -> Board
|      +-- holds -> Symbols (in an NxN grid)
|-- has -> Players (two of them)
+-- tracks -> whose turn it is (a variable, not a class)
```

Build order (bottom-up): **Symbol** → **Player** → **Board** → **Game**.

4. Building the Bricks

Brick 1 — Symbol

A fixed set of values → an enum. The key insight is including **EMPTY** — without it, the board couldn't tell a free cell from a taken one.

Symbol.java

```
public enum Symbol {  
    X,  
    O,  
    EMPTY    // spell it out - clearer than E  
}
```

Why EMPTY drives everything: “is this cell free?” becomes `cell == Symbol.EMPTY`; “board full?” becomes “no cell is EMPTY”. It plays the same role `null` played for the parking spot's vehicle — but an explicit value is safer than nulls.

Brick 2 — Player

Thin by design. A player just holds identity — it doesn't decide moves or check wins. Note: **one class with a symbol field**, not `PlayerX/PlayerO` subclasses, because the two players behave identically and differ only in a value (**data, not behaviour**).

Player.java

```
public class Player {  
    private String name;  
    private Symbol symbol;  
  
    public Player(String name, Symbol symbol) {  
        this.name = name;  
        this.symbol = symbol;  
    }  
  
    public String getName() { return name; }  
    public Symbol getSymbol() { return symbol; }  
}
```

Brick 3 — Board (structure)

A 2D array of Symbols plus the size `N`. The constructor fills every cell with `EMPTY` — a fresh Java array is full of `null`, so we set it explicitly.

Board.java (part 1)

```

public class Board {
    private int size;           // N
    private Symbol[][] grid;

    public Board(int size) {
        this.size = size;
        this.grid = new Symbol[size][size];

        for (int r = 0; r < size; r++) {           // every cell starts EMPTY
            for (int c = 0; c < size; c++) {
                grid[r][c] = Symbol.EMPTY;
            }
        }
    }

    public int getSize() { return size; }
}

```

A design temptation to resist: storing the whole PLayer in each cell. The win-check only ever needs to know “X, O, or empty?” — and the symbol already identifies the player. Storing more is duplicated data with no benefit. **Store the minimum a cell needs.**

Brick 3 — Board (validate & place)

Two checks before placing: the position must be **in bounds**, and the cell must be **empty**. Note these are split into a **checker** and a **doer** — the same check-then-act shape as `canFit()` / `park()` in Parking Lot.

Board.java (part 2)

```
// CHECK: is this move allowed?
public boolean isValidMove(int row, int col) {
    // 1. in bounds? (uses `size`, never a hardcoded 3)
    if (row < 0 || row >= size || col < 0 || col >= size) {
        return false;
    }
    // 2. is the cell empty?
    return grid[row][col] == Symbol.EMPTY;
}

// DO: place the mark (already validated)
public void placeMark(int row, int col, Symbol symbol) {
    grid[row][col] = symbol;
}

// used by the smart bot later to 'try' a move
public void undoMark(int row, int col) {
    grid[row][col] = Symbol.EMPTY;
}
```

Order matters inside `isValidMove`: the bounds check runs *first*, so we never index outside the array when checking if the cell is empty.

Brick 3 — Board (draw check & printing)

Board.java (part 3)

```
// draw = no EMPTY cell left
public boolean isBoardFull() {
    for (int r = 0; r < size; r++) {
        for (int c = 0; c < size; c++) {
            if (grid[r][c] == Symbol.EMPTY) return false;
        }
    }
    return true;
}

public void printBoard() {
    for (int r = 0; r < size; r++) {
        for (int c = 0; c < size; c++) {
            Symbol s = grid[r][c];
            System.out.print((s == Symbol.EMPTY ? "." : s.toString()) + " ");
        }
        System.out.println();
    }
    System.out.println();
}
```

5. The Win Check — $N \times N$, Not Hardcoded

The most interesting method in the problem. A player wins with **N marks in a line**. Count the lines on an $N \times N$ board:

Lines to check	How many	Index trick
Rows	N	walk c across each fixed r
Columns	N	walk r down each fixed c
Main diagonal	1	grid[i][i] — same index twice
Anti-diagonal	1	grid[i][size-1-i]

Total: $2N + 2$ lines. For 3×3 that's 8; for 5×5 it's 12 — it scales with N, which is exactly why nothing can be hardcoded.

The anti-diagonal formula, made concrete for a 3×3 (size = 3):

```
i=0 -> grid[0][3-1-0] = grid[0][2]    (top-right)
i=1 -> grid[1][3-1-1] = grid[1][1]    (centre)
i=2 -> grid[2][3-1-2] = grid[2][0]    (bottom-left)

`size - 1 - i` is worth memorising - it appears in many grid problems.
```

Board.java (part 4) — checkWin

```

public boolean checkWin(Symbol symbol) {
    // 1. all N rows
    for (int r = 0; r < size; r++) {
        boolean allSame = true;
        for (int c = 0; c < size; c++) {
            if (grid[r][c] != symbol) { allSame = false; break; }
        }
        if (allSame) return true;
    }

    // 2. all N columns
    for (int c = 0; c < size; c++) {
        boolean allSame = true;
        for (int r = 0; r < size; r++) {
            if (grid[r][c] != symbol) { allSame = false; break; }
        }
        if (allSame) return true;
    }

    // 3. main diagonal (row == col)
    boolean allSame = true;
    for (int i = 0; i < size; i++) {
        if (grid[i][i] != symbol) { allSame = false; break; }
    }
    if (allSame) return true;

    // 4. anti-diagonal (col = size-1-row)
    allSame = true;
    for (int i = 0; i < size; i++) {
        if (grid[i][size - 1 - i] != symbol) { allSame = false; break; }
    }
    if (allSame) return true;

    return false;    // no winning line
}
}

```

Every loop uses size, never 3. Change the board to 5×5 and this method works unchanged — that's the whole point of the N×N discipline.

6. Brick 4 — Game, and Running It

The controller. It holds the board and players, and runs the turn loop until someone wins or it's a draw.

Game.java

```
import java.util.Scanner;

public class Game {
    private Board board;
    private Player[] players;
    private int currentPlayerIndex;

    public Game(int boardSize, Player p1, Player p2) {
        this.board = new Board(boardSize);
        this.players = new Player[]{p1, p2};
        this.currentPlayerIndex = 0;    // player 1 starts
    }

    public void play() {
        Scanner scanner = new Scanner(System.in);

        while (true) {
            // 1. whose turn
            Player current = players[currentPlayerIndex];
            System.out.println(current.getName() + "'s turn ("
                               + current.getSymbol() + "). Enter row and col:");

            // 2. they pick a spot
            int row = scanner.nextInt();
            int col = scanner.nextInt();

            // 3. validate
            if (!board.isValidMove(row, col)) {
                System.out.println("Invalid move! Try again.");
                continue;    // SAME player retries - never reaches step 7
            }

            // 4. place
            board.placeMark(row, col, current.getSymbol());
            board.printBoard();

            // 5. check WIN first
            if (board.checkWin(current.getSymbol())) {
                System.out.println(current.getName() + " wins!");
                break;
            }

            // 6. then check draw
            if (board.isBoardFull()) {
                System.out.println("It's a draw!");
                break;
            }

            // 7. switch player
            currentPlayerIndex = 1 - currentPlayerIndex;
        }
        scanner.close();
    }
}
```

Three details worth knowing

- 1. `continue` makes the same player retry.** It jumps back to the loop start, skipping the switch-player line at the bottom. An `if/else` that fell through would wrongly pass the turn.
- 2. Win check must come BEFORE draw check.** The last cell could complete a winning line — the board is then both full *and* won. Checking draw first would wrongly declare a tie.
- 3. The switch trick:** `currentPlayerIndex = 1 - currentPlayerIndex` flips `0↔1` forever. (For more players: `(i + 1) % players.length`.)

Running it

Main.java

```
public class Main {
    public static void main(String[] args) {
        Player p1 = new Player("Syamala", Symbol.X);
        Player p2 = new Player("Sumanth", Symbol.O);

        Game game = new Game(3, p1, p2);    // 3x3 board
        game.play();
    }
}
```

OUTPUT

```
Syamala's turn (X). Enter row and col:
0 0
X . .
. . .
. . .

Sumanth's turn (O). Enter row and col:
1 1
X . .
. O .
. . .

...
Syamala wins!
```

Want a 5×5 board? Change one number: `new Game(5, p1, p2)`. Everything else works unchanged.

The complete class list

Symbol (enum), Player, Board, Game, Main — four small classes plus a runner. Clean, no over-engineering, works for any N.

7. Extension — “Now Add a Computer Player”

This is the interviewer checking whether your design absorbs change cleanly. First, apply the **data-vs-behaviour test**:

Human: waits for keyboard input. **Computer:** scans the board and computes a cell.
Genuinely different *logic* → **behaviour** → separate classes. ✓

But note *what* differs: not the whole player — a bot still has a name, a symbol, still takes turns. **Only the move-selection varies.** That's the signature of Strategy: extract just the varying behaviour.

Step 1 — a Move holder + the strategy interface

MoveStrategy.java

```
// Java has no tuples, so a tiny class to return two numbers
public class Move {
    private int row, col;
    public Move(int row, int col) { this.row = row; this.col = col; }
    public int getRow() { return row; }
    public int getCol() { return col; }
}

public interface MoveStrategy {
    Move selectMove(Board board);    // given a board, give me a move
}
```

Step 2 — the concrete strategies

The three strategies

```

import java.util.Scanner;

public class HumanMoveStrategy implements MoveStrategy {
    private Scanner scanner = new Scanner(System.in);

    public Move selectMove(Board board) {
        System.out.print("Enter row and col: ");
        return new Move(scanner.nextInt(), scanner.nextInt());
    }
}

public class RandomComputerStrategy implements MoveStrategy {
    public Move selectMove(Board board) {
        for (int r = 0; r < board.getSize(); r++) {
            for (int c = 0; c < board.getSize(); c++) {
                if (board.isValidMove(r, c)) { // reuse Board's own check!
                    System.out.println("Computer plays: " + r + " " + c);
                    return new Move(r, c);
                }
            }
        }
        return null;
    }
}

public class SmartComputerStrategy implements MoveStrategy {
    private Symbol mySymbol;
    public SmartComputerStrategy(Symbol mySymbol) { this.mySymbol = mySymbol; }

    public Move selectMove(Board board) {
        int n = board.getSize();
        // 1. can I win right now? take it.
        for (int r = 0; r < n; r++) {
            for (int c = 0; c < n; c++) {
                if (board.isValidMove(r, c)) {
                    board.placeMark(r, c, mySymbol); // try it
                    boolean wins = board.checkWin(mySymbol);
                    board.undoMark(r, c); // take it back
                    if (wins) return new Move(r, c);
                }
            }
        }
        // 2. otherwise first available
        for (int r = 0; r < n; r++)
            for (int c = 0; c < n; c++)
                if (board.isValidMove(r, c)) return new Move(r, c);
        return null;
    }
}

```

Each class has genuinely different logic — not empty bodies. That confirms it's behaviour, not data, so Strategy is justified. Also note the bots **reuse** `board.isValidMove()` rather than reimplementing “is this cell free” — same delegation discipline as Parking Lot.

Step 3 — plug the strategy into Player

Player.java (updated)

```
public class Player {
    private String name;
    private Symbol symbol;
    private MoveStrategy moveStrategy;    // the plugged-in behaviour

    public Player(String name, Symbol symbol, MoveStrategy moveStrategy) {
        this.name = name;
        this.symbol = symbol;
        this.moveStrategy = moveStrategy;
    }

    // the player DELEGATES move-picking to its strategy
    public Move makeMove(Board board) {
        return moveStrategy.selectMove(board);
    }

    public String getName()    { return name; }
    public Symbol getSymbol() { return symbol; }
}
```

Step 4 — the Game loop barely changes

Two lines changed

```
// BEFORE - hardcoded keyboard input
int row = scanner.nextInt();
int col = scanner.nextInt();

// AFTER - ask the player, whoever they are
Move move = current.makeMove(board);
int row = move.getRow();
int col = move.getCol();
```

Everything else — validate, place, check win, check draw, switch — is **completely untouched**. The Game has no idea whether it's running human-vs-human, human-vs-bot, or bot-vs-bot.

Step 5 — the payoff

Main.java (extended)

```

public class Main {
    public static void main(String[] args) {
        // human vs human
        Player p1 = new Player("Syamala", Symbol.X, new HumanMoveStrategy());
        Player p2 = new Player("Sumanth", Symbol.O, new HumanMoveStrategy());

        // human vs computer - just swap the strategy!
        Player human = new Player("Syamala", Symbol.X, new HumanMoveStrategy());
        Player bot   = new Player("Computer", Symbol.O, new RandomComputerStrategy());

        // computer vs computer - watch two bots play
        Player bot1 = new Player("Bot1", Symbol.X, new RandomComputerStrategy());
        Player bot2 = new Player("Bot2", Symbol.O, new SmartComputerStrategy(Symbol.O));

        Game game = new Game(3, human, bot);
        game.play();
    }
}

```

Three completely different game modes, and Game never changed. Want a minimax hard-mode bot? Write `MinimaxStrategy` implements `MoveStrategy`, plug it in, touch nothing else.

8. Why Strategy — and the Bigger Lesson

What you'd say aloud in the interview

“Only the *move-selection* differs between human and computer — everything else about a player is identical. So instead of subclassing `Player` into `Human/Computer`, I extracted just that varying behaviour into a `MoveStrategy`. Now I can add new bot difficulties by writing one new class, without touching `Player` or `Game`.”

Why not subclass `Player`?

✗ INHERITANCE

```
// the tempting alternative
class HumanPlayer extends Player { ... }
class ComputerPlayer extends Player { ... }

// but then: easy / medium / hard bots?
class EasyComputerPlayer extends Player { ... }
class MediumComputerPlayer extends Player { ... }
class HardComputerPlayer extends Player { ... } // class explosion
```

With Strategy those are just three `MoveStrategy` implementations plugged into the *same* `Player`.

Composition over inheritance — a phrase interviewers love hearing.

The SOLID connection

Principle	Where it shows up
Open/Closed	add a bot difficulty by writing a new strategy; never edit <code>Player</code> or <code>Game</code>
Dependency Inversion	<code>Player</code> depends on the <code>MoveStrategy</code> INTERFACE, injected in the constructor
Single Responsibility	<code>Player</code> holds identity; the strategy owns move-picking logic

The big lesson

Look at what happened across the two versions:

Version	Patterns used	Why
Base Tic Tac Toe	NONE	no genuine variation — a pattern would be over-engineering
+ computer player	Strategy	a requirement created real behavioural variation

You don't decide “I'll use Strategy” upfront. You design simply, and when a requirement introduces real variation, the pattern *arrives on its own*. The extension question is the interviewer checking whether your original design could absorb change cleanly — and this one did, with **two lines changed** in the game loop.

That's the discipline: a clean 4-class solution with clear responsibilities beats a pattern-stuffed one every time. Knowing when **not** to reach for a pattern is exactly what separates a good design from a cluttered one.